Intelligence artificielle générative

Redha Moulla

Paris, 5-6 novembre 2026

Plan de la formation

- Qu'est-ce que l'intelligence artificielle ?
- Eléments de machine learning et de deep learning
- Large Language Models (LLM)
- Les bonnes pratiques de prompting
- Les RAG et les Agents
- L'IAG dans le cycle de développement logiciel
- Enjeux éthiques, de sécurité et de conformité de l'IAG

Qu'est-ce que l'intelligence artificielle ?

Définition littérale de l'intelligence artificielle

1. Intelligence

Ensemble des fonctions mentales ayant pour objet la connaissance conceptuelle et relationnelle.

- Larousse

2. Artificielle

Qui est produit de l'activité humaine (opposé à la nature).

- Larousse

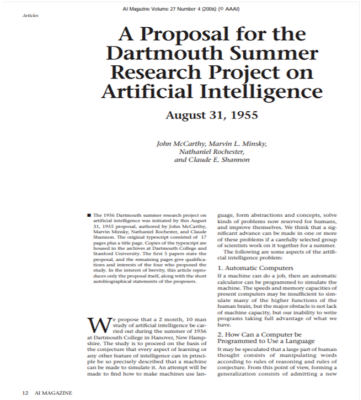
Qu'est-ce que l'intelligence ?

La notion d'intelligence recouvre plusieurs facultés cognitives :

1. **Raisonnement** : La capacité à résoudre des problèmes et à faire des déductions logiques.
2. **Apprentissage** : L'aptitude à acquérir de nouvelles connaissances et à s'améliorer grâce à l'expérience.
3. **Perception** : La compétence pour reconnaître et interpréter les stimuli sensoriels.
4. **Compréhension** : L'habileté à saisir le sens et l'importance de divers concepts et situations.
5. **Mémorisation** : La faculté de stocker et de rappeler des informations.
6. **Créativité** : Le pouvoir d'inventer ou de produire de nouvelles idées, de l'originalité dans la pensée.

Mais est-ce que l'intelligence est réductible à des facultés mesurables ?

Conférence de Dartmouth ?



AI Magazine Volume 27 Number 4 (2006) (© AAAI)

Articles

A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence

August 31, 1955

John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon

■ The 1956 Dartmouth summer research project on artificial intelligence was initiated by this August 31, 1955 proposal, authored by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. The original typescript consisted of 17 pages plus a title page. Copies of the typescript are housed in the archives at Dartmouth College and Stanford University. The first page of the original proposal and the remaining pages give qualifications and interests of the four who proposed the study. In the interest of brevity, this article reproduces only the proposal itself, along with the short autobiographical statements of the proposers.

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

The following are some aspects of the artificial intelligence problem:

1. Automatic Computers

If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. The speeds and memory capacities of present computers may be insufficient to simulate many of the higher functions of the human brain, but the major obstacle is not lack of machine capacity, but our inability to write programs taking full advantage of what we have.

2. How Can a Computer be Programmed to Use a Language

It may be speculated that a large part of human thought consists of manipulating words according to rules of reasoning and rules of conjecture. From this point of view, forming a generalization consists of admitting a new

"We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. *The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer."*

L'intelligence artificielle selon John McCarthy

"It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable."

— John McCarthy

Le Test de Turing

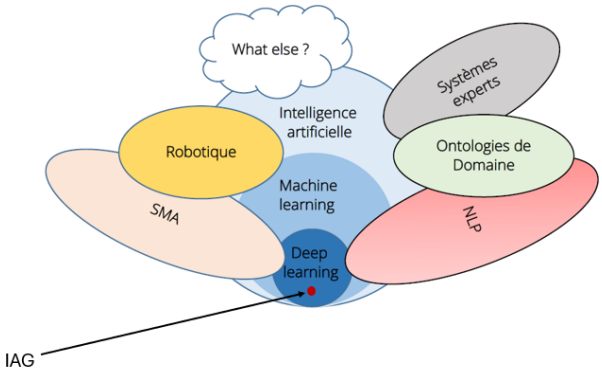
Le Test de Turing, développé par Alan Turing en 1950, est une tentative de mesurer l'intelligence d'une machine, plus précisément de la factulté d'une machine à penser. Cette dernière n'étant pas si évidente à mesurer, le test substitue finalement à la faculté de penser celle de traiter le langage naturel comme un humain.

Les points clés du Test de Turing sont :

- Un interrogateur humain engage une conversation avec un humain et une machine, chacun étant caché de la vue de l'interrogateur.
- Si l'interrogateur ne peut pas déterminer systématiquement quelle est la machine, celle-ci est considérée comme ayant passé le test.
- Le test ne mesure pas la connaissance ou la capacité à être véridique, mais plutôt la capacité de reproduire le comportement humain.

Définition pragmatique de l'intelligence artificielle

Il s'agit d'un ensemble de techniques qui permettent à la machine d'accomplir des tâches qui requièrent traditionnellement une intelligence humaine.



IA forte vs IA faible

La distinction entre IA forte et IA faible se réfère à deux approches conceptuelles différentes dans le domaine de l'intelligence artificielle.

IA faible :

- Aussi connue sous le nom d'IA "étroite", elle est conçue pour effectuer des tâches spécifiques et ne possède pas de conscience.
- Les systèmes d'IA faible agissent et réagissent uniquement en fonction des instructions programmées et des algorithmes spécifiques.
- Exemples : assistants virtuels, systèmes de recommandation, reconnaissance vocale.

IA forte :

- Vise à créer des machines dotées de capacité de généralisation à l'image de l'intelligence humaine.
- L'IA forte serait capable d'apprendre, de raisonner, de résoudre des problèmes et de prendre des décisions indépendamment.
- À ce jour, l'IA forte reste un objectif à atteindre, qui fait l'objet de recherches intensives.

Eléments de machine learning et de deep learning

Qu'est-ce que la machine learning ?

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

- **IA discriminative** : Modélise la frontière entre les classes.
  Exemples : classification de commentaires clients, routage de mails, credit scoring, diagnostic médical, etc.
- **IA générative** : modélise la distribution des données pour en générer de nouvelles.
  Exemples : Génération de texte, d'images, de musique, etc.
- **Apprentissage par renforcement (Reinforcement Learning)** : apprend à prendre des décisions séquentielles dans un environnement.
  L'agent n'imite pas ou ne classe pas, il agit pour maximiser une récompense. *Exemples :* Algorithmes d'enchères, jeux vidéos, etc.

Machine learning discriminatif

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

Le machine learning englobe deux grandes familles d'apprentissage :

- **Supervisé** : Les algorithmes apprennent à partir de données étiquetées pour faire des prédictions ou classifications.

- **Non supervisé** : L'apprentissage est effectué sur des données non étiquetées pour trouver des structures cachées.

L'apprentissage supervisé

L'apprentissage supervisé consiste à apprendre un modèle qui associe une étiquette (*label*) à un ensemble de caractéristiques (*features*).

- **Inputs** : un jeu de données *annotées* pour entraîner le modèle.
  - Exemple : des textes (tweets, etc.) avec les *sentiment* associés, positifs ou négatifs.
- **Output** : une étiquette pour un point de donnée inconnu par le modèle.

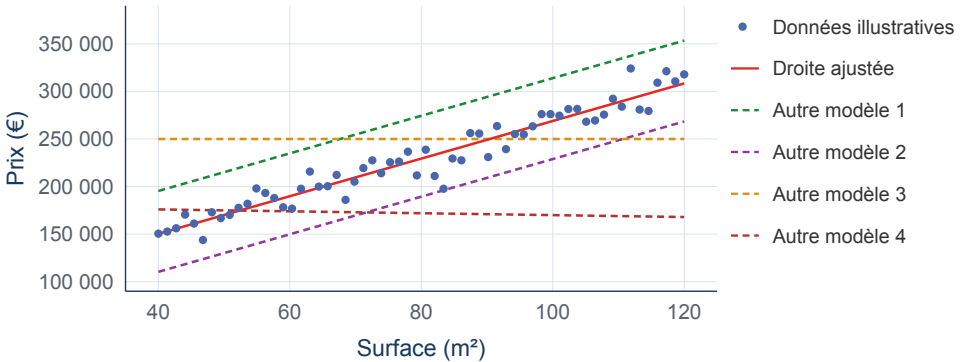
L'apprentissage supervisé se décline lui-même en deux grandes familles :

- **La classification** : prédire une catégorie ou une classe.
  - Exemple : prédire l'étiquette d'une image (chat, chien, etc.), le sentiment associé à un texte, le centre d'intérêt d'un client à partir de ses commentaires, etc.
- **La régression** : prédire une valeur continue (un nombre réel typiquement).
  - Exemple : prédire le prix d'un appartement, la lifetime value d'un client, etc.

Régression linéaire simple

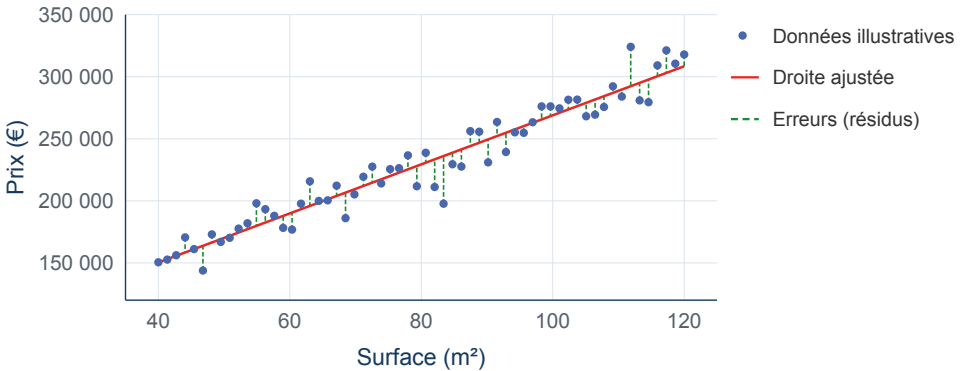
Soit un ensemble de n obeservations $x_1, x_2, ..., x_n$ avec les labels correspondants $y_1, y_2, ..., y_n$, on cherche le modèle linéaire qui ajuste le mieux ces données.

$$\hat{y} = \beta_0 + \beta_1 x$$

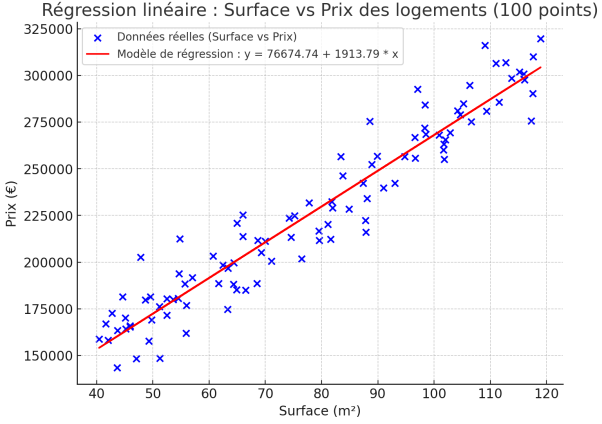


Minimisation du risque empirique 1/2

L'erreur de prédiction pour la i ième observation est : $e_i = y_i - \hat{y}_i$. où $\hat{y}_i = \beta_0 + \beta_1 x_i$.



Prédire le prix en fonction de la surface : inférence



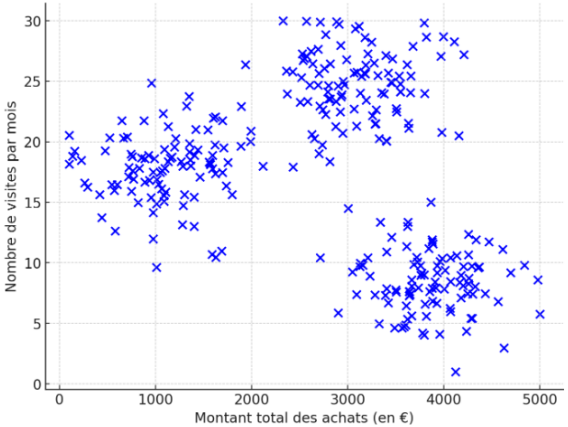
L'apprentissage non supervisé

L'apprentissage non supervisé se réfère à l'utilisation de modèles d'apprentissage automatique pour identifier des patterns et des structures dans des données qui ne sont pas étiquetées.

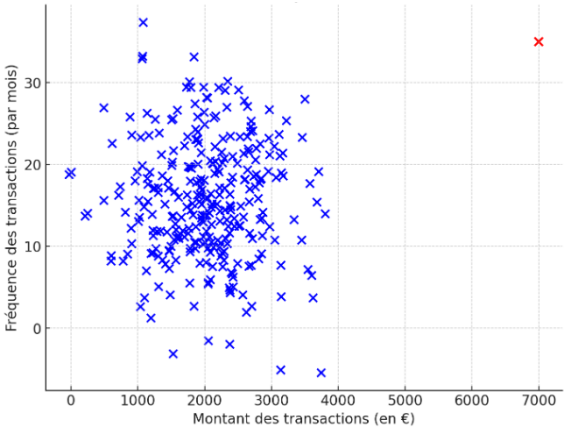
Principales typologies de l'apprentissage non supervisé :

- **Clustering** : Regroupement de points de données similaires ensemble. Exemple : segmentation de marché, regroupement social.
- **Détection d'anomalies** : Détecter des observations dont les caractéristiques sont inhabituelles par rapport à la majorité.

Exemple de clustering : segmentation clients



Exemple de détection d'anomalies : fraude bancaire



Eléments de deep learning

Brève histoire du deep learning

1940-1980 : Fondations

- 1943 : Neurone artificiel (McCulloch & Pitts).
- 1986 : Rétropropagation (Rumelhart, Hinton, Williams).

1990-2000 : Hiver

- Manque de données et puissance de calcul.
- Progrès en théorie : SVM, modèles bayésiens.

2010-2020 : Renaissance

- 2012 : AlexNet révolutionne la vision par ordinateur.

Depuis 2020 : Multimodalité et LLM

- 2017 : Transformer, base des LLM.
- 2018 - 2020 : GPT-3, BERT.
- 2022 : ChatGPT...

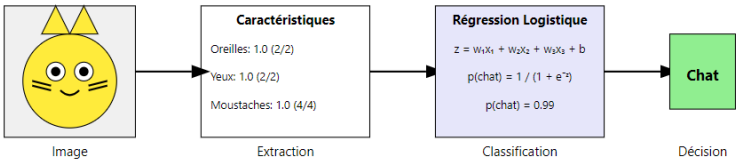
Introduction aux réseaux de neurones

Définition : Les réseaux de neurones sont des modèles computationnels inspirés par le fonctionnement des neurones dans le cerveau humain. Ils sont capables d'apprendre des tâches complexes en modélisant des relations non linéaires entre les entrées et les sorties.

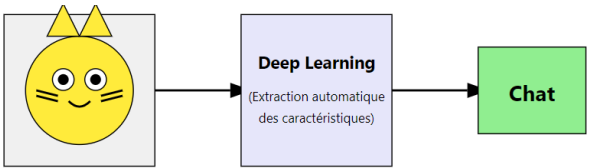
Caractéristiques :

- **Extraction automatique des features** : Capacité d'adaptation et d'extraction des features à partir des données sans programmation explicite.
- **Modélisation non linéaire** : Aptitude à capturer des relations complexes dans les données.
- **Modélisation en grande dimension** : Les modèles de deep learning sont particulièrement adaptés pour les données en grande dimension (images, texte, etc.).
- **Flexibilité** : Applicables à un large éventail de tâches et de typologies de données (images, langage naturel, données graphiques, etc.).

Extraction de features en machine learning "classique"



Extraction de features en deep learning



Réseau de neurones et neurone artificiel

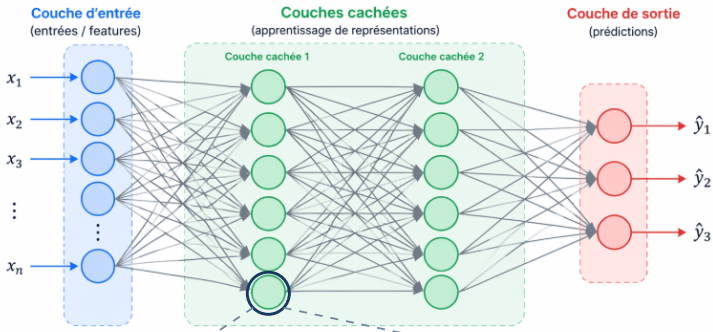
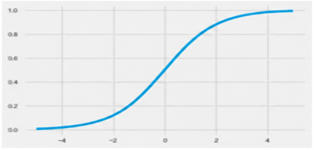
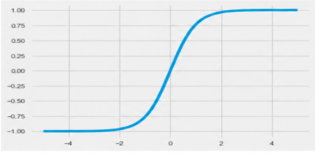


Illustration des fonctions d'activation

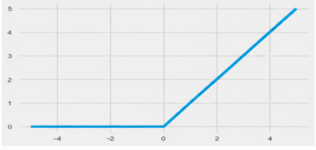
Sigmoïde



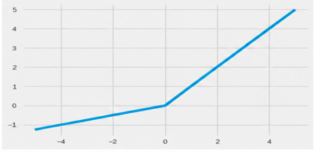
Tanh



ReLU



ReLU paramétrique



La fonction SoftMax dans la classification dans la dernière couche

Le SoftMax transforme une liste de scores en **probabilités de classes**. C'est la dernière étape d'un modèle de classification multiclasses.

Formule :

$$P(y_i) = \frac{e^{s_i}}{\sum_{j=1}^{K} e^{s_j}}$$

où s_i représente le score associé à la classe i, et K le nombre total de classes.

Exemple :

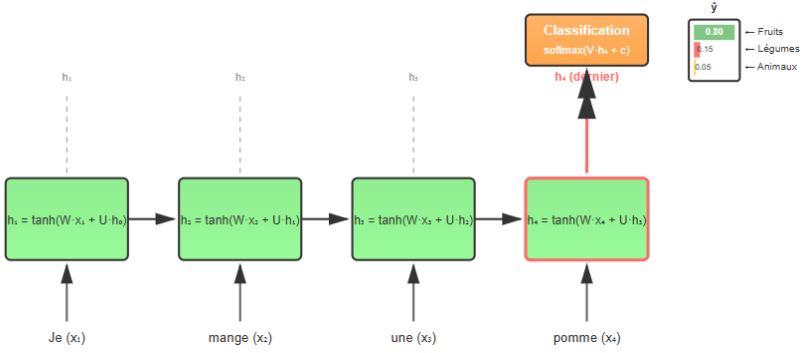
$$\text{Image} \rightarrow \begin{cases} \text{chat} : 0.80 \\ \text{chien} : 0.10 \\ \text{mouton} : 0.10 \end{cases}$$

$\rightarrow$ Le modèle prédit la classe la plus probable (*chat*), tout en conservant une estimation pour les autres.

Large Language Models (LLMs)

Anciens modèles pour le traitement du langage : RNN

- **Mémoire à court terme** : Les reaseaux de neurones récurrents (RNN) ont la capacité de se "souvenir" d'informations passées grâce à leurs connexions récurrentes.
- **Traitement de séquences** : Ils sont particulièrement adaptés pour des tâches où les données sont séquentielles et où le contexte est important.



Transformers



arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Attention Is All You Need

Ashish Vaswani[*] Google Brain avaswani@google.com

Noam Shazeer[*] Google Brain noam@google.com

Niki Parmar[*] Google Research nikip@google.com

Jakob Uszkoreit[*] Google Research usz@google.com

Llion Jones[*] Google Research llion@google.com

Aidan N. Gomez[* †] University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser[*] Google Brain lukaszkaiser@google.com

Illia Polosukhin[* ‡] illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

[*]Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Transformer originel

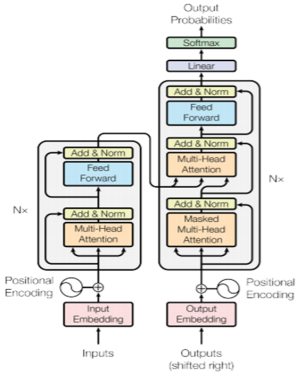
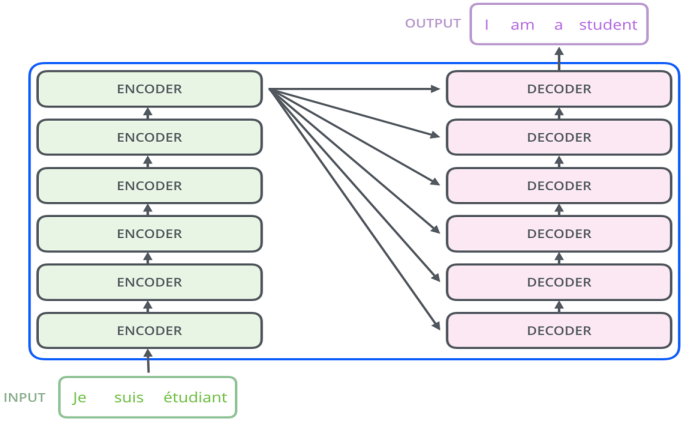
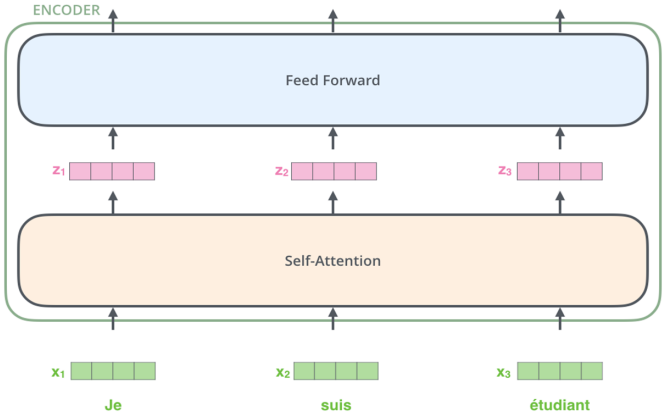


Figure 1: The Transformer - model architecture.

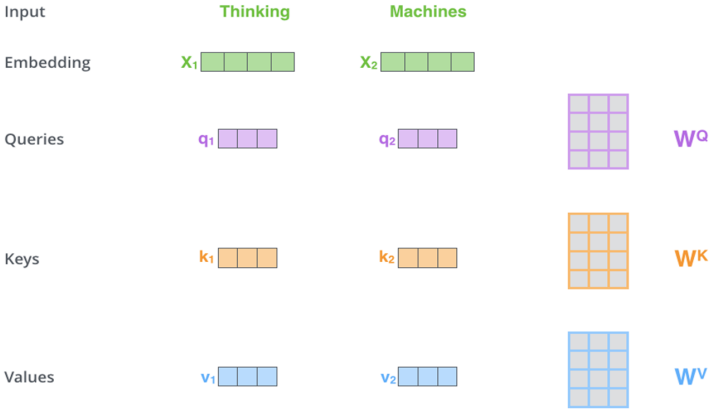
Mécanisme de cross-attention



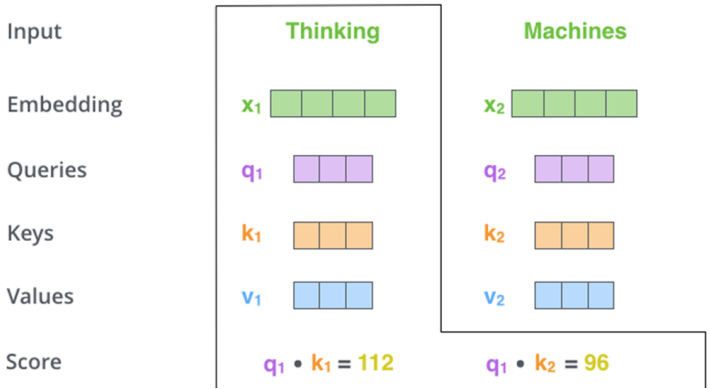
Mécanisme de self-attention



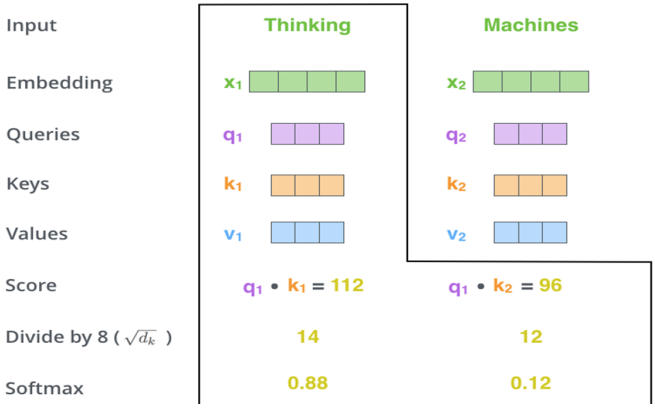
Fonctionnement du mécanisme de self-attention



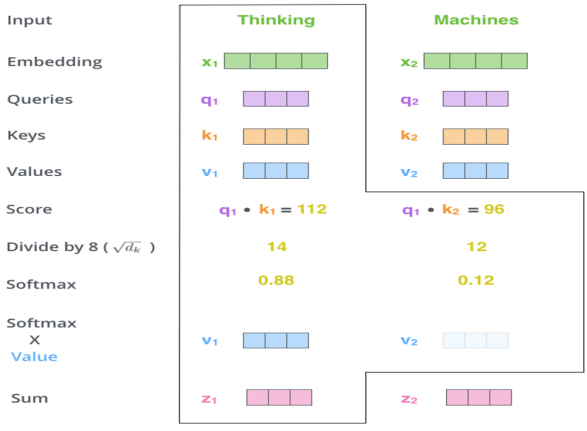
Calcul des scores de self-attention



Calcul des scores de self-attention



Calcul de la nouvelle représentation



Performances du tranformer originel

Entraîné sur WMT 2014 English- German dataset, comprenant près de 4.5 millions de phrases sentence, le WMT 2014 English-French dataset, comprenant près de 36 millions de phrases.

- 8 NVIDIA P10 GPUs
- **Base** : 12 heures
- **Large** : 3.5 jours

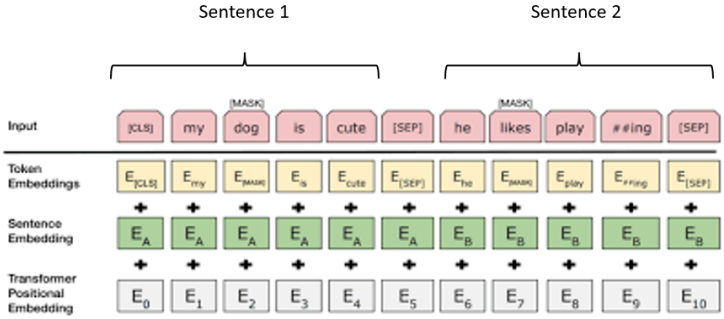
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

BERT

BERT (Bidirectional Encoder Representations from Transformers) représente une avancée majeure dans le NLP, introduisant une approche novatrice pour la modélisation de langage. Il utilise la bidirectionnalité pour comprendre le contexte des mots, permettant une compréhension plus fine du langage.

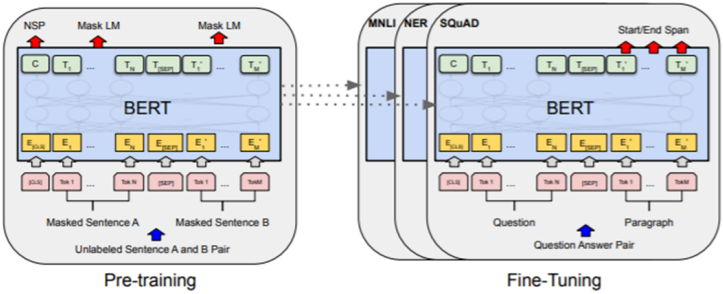
Pré-entraînement



Fine-tuning de BERT pour des tâches spécifiques

Après pré-entraînement, BERT est affiné pour des tâches spécifiques :

- Ajout d'une couche de sortie spécifique à la tâche (classification, NER, QA).
- Fine-tuning de tous les paramètres du modèle pré-entraîné sur le corpus de la tâche.



Performances de BERT

BERT a été pré-entrainé sur le BookCorpus (800 millions de mots) et English Wikipedia (2,500 millions de mots).

Deux modèles :

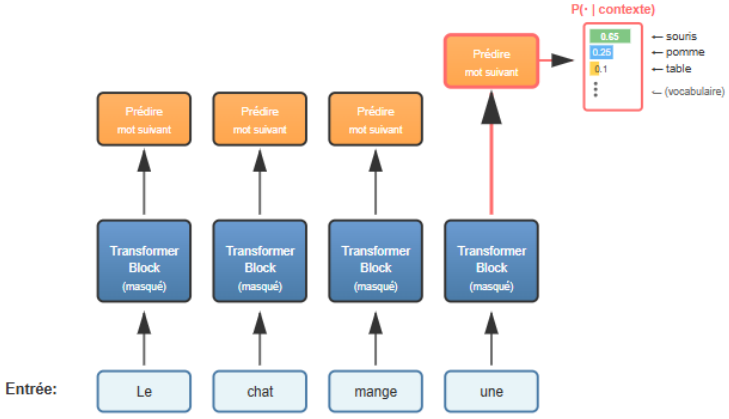
- **BERT Base** : 12 couches avec 110 millions de paramètres.
- **BERT Large** : 24 couches avec 340 millions de paramètres.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

« Ce que je ne peux pas créer,
je ne le comprends pas. »

– Richard Feynman

Fonctionnement des modèles génératifs



Le paramètre de température

Définition La température (T) contrôle le degré d'aléa dans les réponses générées. Elle agit sur la distribution de probabilité des mots avant l'échantillonnage.

Formule mathématique :

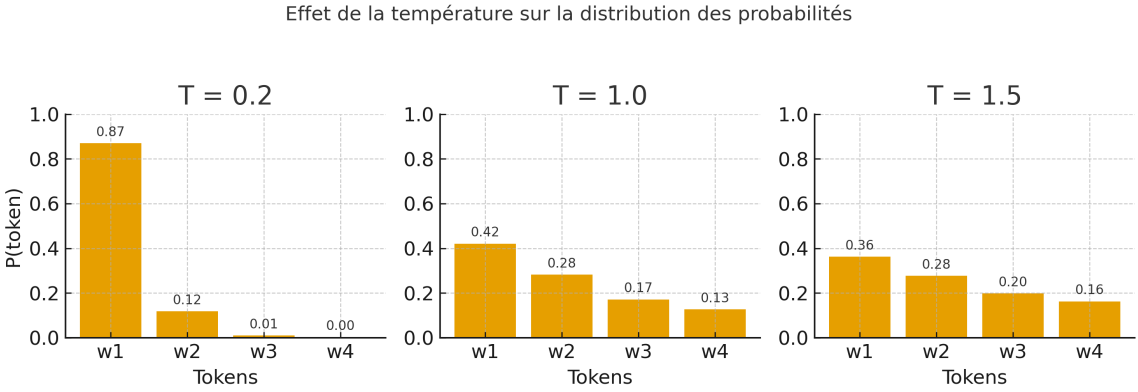
$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T < 1 \rightarrow$ distribution plus concentrée, réponses plus prévisibles
- $T > 1 \rightarrow$ distribution plus plate, réponses plus variées et créatives

Autrement dit :

- Basse température $\rightarrow$ précision et cohérence.
- Haute température $\rightarrow$ imagination et diversité.

Illustration graphique de l'effet de la température



La tokénisation

Problème de départ : Les modèles de langage ne peuvent pas traiter directement du texte brut : les mots, les accents, la ponctuation et les variations sont trop nombreux (plusieurs millions de combinaisons possibles).

La tokénisation : Elle consiste à découper le texte en **unités réutilisables et limitées en nombre**, appelées *tokens*. Chaque token est ensuite représenté par un identifiant numérique que le modèle peut apprendre à prédire.

Intérêt de la tokénisation :

- Réduire la complexité du vocabulaire : on passe de millions de mots à quelques dizaines de milliers de tokens.
- Gérer les mots inconnus ou rares en les décomposant en sous-unités connues.
- Offrir une représentation numérique uniforme du langage, compatible avec les réseaux de neurones.

La composition des données d'entraînement des modèles

Principe général Les modèles de langage sont entraînés sur d'immenses corpus de texte — souvent plusieurs milliers de milliards de tokens — issus de sources très diverses pour couvrir la langue, les faits et les styles.

Sources typiques de données :

- **Web public** : pages web, articles, forums, Wikipédia (ex. Common Crawl).
- **Livres et documents** : œuvres littéraires, articles scientifiques, rapports techniques.
- **Données de code** : dépôts GitHub, notebooks, documentation technique.
- **Conversations et dialogues** : jeux de données de chat, discussions de forums, données annotées par des humains.
- **Données synthétiques** : textes générés par d'autres modèles pour enrichir certaines thématiques.

Équilibre recherché :

- Diversité linguistique et thématique (langues, registres, domaines).
- Qualité et cohérence (filtrage du spam, des doublons, des contenus toxiques).
- Représentation équilibrée entre langage courant, technique et conversationnel.

GPT-3

- Développé par OpenAI, publié en 2020.
- Modèle auto-régressif basé sur l'architecture Transformer.

Caractéristique	**Valeur / Détail**
Date	2020
Paramètres	175 milliards
Corpus d'entraînement	Environ 300 milliards de tokens
Contexte maximum	2048 tokens
Ressources GPU	Plusieurs centaines de GPU (type V100)
Coût d'entraînement (estim.)	4,6 M$ à 12 M$ (selon les sources)

- **Applications** : génération de texte, Q&R, résumé, traduction, assistance à la programmation.

Le few-shot learning

- Les grands modèles (comme GPT-3) peuvent réaliser des tâches **sans être explicitement réentraînés** dessus.
- Le principe repose sur **l'utilisation de quelques exemples** (exemples étiquetés) directement dans le prompt ou l'entrée.
- **Zero-shot** : aucune démonstration fournie, le modèle doit comprendre la tâche à partir de sa connaissance générale.
- **One-shot / Few-shot** : on inclut un nombre très réduit d'exemples (un ou quelques-uns) pour guider le modèle dans la résolution de la tâche.
- Exemple :

English : "cat" $\rightarrow$ French : "chat"
English : "house" $\rightarrow$ French : "maison"
English : "car" $\rightarrow$ French : "voiture"

English : "tree" $\rightarrow$ French :

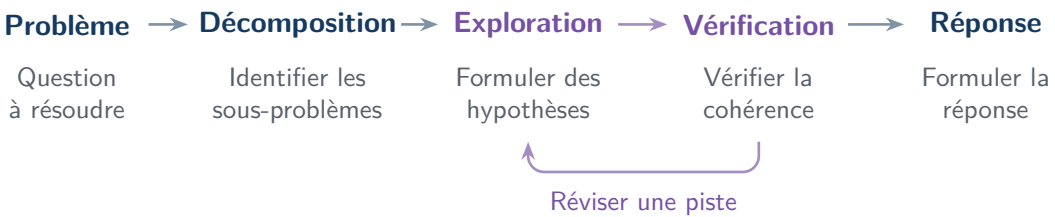
ChatGPT

ChatGPT, développé par OpenAI, est un modèle de langage basé sur l'architecture GPT (Generative Pre-trained Transformer) optimisé pour comprendre et générer des dialogues naturels. L'entraînement de ChatGPT se décline selon les étapes suivantes :

1. **Pré-entraînement sur un corpus volumineux :** Comme GPT-3, ChatGPT est d'abord pré-entraîné sur un vaste ensemble de données textuelles, englobant un large éventail de la littérature disponible sur Internet, pour apprendre une compréhension générale du langage.
2. **Fine-tuning supervisé :** Ensuite, ChatGPT est affiné sur des dialogues spécifiques pour améliorer ses compétences conversationnelles. Cette étape utilise des paires question-réponse et des conversations pour enseigner au modèle des structures de dialogue et des réponses contextuellement appropriées.
3. **Reinforcement Learning from Human Feedback (RLHF) :** Utilisation de techniques de renforcement pour ajuster les réponses du modèle basées sur les préférences et les corrections fournies par des évaluateurs humains, raffinant davantage la pertinence et la naturalité des réponses.

Modèles de raisonnement

Ces modèles sont entraînés à décomposer un problème, explorer des pistes et vérifier leurs résultats avant de répondre.



Davantage de calcul peut améliorer la résolution des problèmes complexes, au prix d'un temps de réponse plus long.

LLMs propriétaires vs Open Source

- **Modèles propriétaires**
  - ChatGPT, Claude, Gemini, Grok...
  - Poids non disponibles au public.
  - Performances élevées, accès limité par API payante
  - Protection intellectuelle stricte (code non accessible)
  - Support technique assuré, documentation avancée
  - Dépendance aux fournisseurs et coûts potentiellement élevés

- **Modèles open source**
  - Llama, Mistral, Gemma, DeepSeek, Qwen...
  - Poids disponibles au public
  - Transparence du code, flexibilité d'adaptation
  - Performances parfois inférieures, mais amélioration continue
  - Gratuit, mais coût de l'infrastructure à gérer
  - Exposition aux failles potentielles de sécurité, support communautaire

Conclusion : Choisir entre contrôle, flexibilité et performances optimales.

Introduction au prompting

Définition : Le prompting est l'art de formuler des instructions à un modèle de langage afin d'obtenir une réponse pertinente et de qualité.

Idée clé : La qualité de la sortie dépend fortement de la manière dont l'entrée est rédigée.

Enjeux :

- Obtenir des résultats précis et cohérents.
- Réduire les risques d'ambiguïtés et d'hallucinations.
- Adapter le style et le niveau de détail de la réponse.

Techniques de base du prompting

- **Prompt clair et explicite :** éviter les formulations vagues.
- **Contexte :** fournir des informations supplémentaires pour orienter le modèle.
- **Format attendu :** indiquer la structure de sortie souhaitée (liste, tableau, texte court).
- **Exemples :** montrer un ou plusieurs cas pour guider la génération.

Exemple : Au lieu de "C'est quoi le machine learning?", écrire "Donne une explication détaillée du fonctionnement du machine learning, illustrée avec un exemple".

1. Le ton change la structure du raisonnement

Principe : Le ton du prompt influence non seulement le style, mais aussi la structure logique de la réponse.

Effets observés

- **Ton calme et analytique :** produit une écriture déductive et ordonnée, où le modèle explicite les causes et développe les idées avant de conclure.
- **Ton urgent ou tendu :** génère une écriture télégraphique, orientée vers l'action, avec phrases courtes et listes de décisions.

Exemples

« Explique posément, en suivant un raisonnement logique. »
« Vite : résume-moi ce que je dois dire au client dans deux minutes. »

À retenir : Le ton est un levier cognitif : il modifie la manière dont le modèle raisonne avant même d'écrire.

2. Les métaphores encadrent la pensée

Principe : La métaphore impose un cadre de raisonnement : elle force le modèle à réencoder le concept dans une structure familière.

Effets observés

- Elle permet d'aborder des notions abstraites par transfert analogique.
- Elle structure naturellement la réponse selon les dimensions de la métaphore (par exemple : ingrédients, recettes, cuisson).

Exemples

« Explique le machine learning comme si c'était une cuisine. »
« Explique l'optimisation comme une randonnée avec des étapes et des sommets. »

À retenir : La métaphore agit comme un cadre cognitif qui oriente la logique et le vocabulaire du texte.

3. Les rôles activent des corpus latents

Principe : Attribuer un rôle au modèle sélectionne une manière de parler et de raisonner issue de son corpus.

Effets observés

- **Historien :** met l'accent sur la chronologie et les causes profondes.
- **Ingénieur sceptique :** insiste sur les hypothèses, les tests et les limites.
- **Poète critique :** privilégie les images, les tensions et le rythme.

Exemples

« Agis comme un historien en expliquant les causes profondes. »
« Réponds comme un ingénieur qui doute de ses propres hypothèses. »

À retenir : Le rôle agit comme un filtre stylistique et culturel qui oriente la réponse sans en changer le fond.

4. La précision du contexte vaut plus que la longueur

Principe : Un contexte précis et incarné produit de meilleures réponses qu'une longue liste d'instructions abstraites.

Effets observés

- Le modèle comprend mieux la situation lorsqu'elle est ancrée dans un cadre concret.
- Les réponses deviennent plus pertinentes et plus naturelles.

Exemples

« Tu écris à ton supérieur, ton ferme mais respectueux,
objectif : obtenir une validation avant 18 h. »
« En 200 mots, identifie trois risques et formule une demande de décision. »

À retenir : Un cadre incarné aide le modèle à raisonner comme dans une situation réelle.

5. Les verbes d'action changent la dynamique

Principe : Le verbe choisi détermine le type de raisonnement attendu.

Effets observés

- **Explique :** logique de cause à effet.
- **Montre :** logique d'exemple et d'illustration.
- **Révèle :** logique d'enquête ou de découverte.

Exemples

« Explique en trois causes principales. »
« Montre deux exemples concrets et un contre-exemple. »
« Révèle l'élément caché qui change la conclusion. »

À retenir : Le verbe oriente la structure du texte et la nature du raisonnement.

6. Le silence comme outil

Principe : Ne pas tout préciser laisse au modèle une marge d'interprétation, propice à la créativité.

Effets observés

- Le modèle complète avec des régularités contextuelles, souvent pertinentes.
- Trop de contraintes bloque la pensée et conduit à des réponses formatées.

Exemples

« Donne trois options, dont une plus audacieuse, sans les justifier. »
« Rédige un brouillon brut, sans titres, avec les idées clés uniquement. »

À retenir : Le vide agit comme un espace de respiration qui stimule la créativité.

7. Les émotions modulent l'énergie du contenu

Principe : Le ton émotionnel du prompt influence le rythme et la profondeur de la réponse.

Effets observés

- **Colère :** texte direct, tranchant, orienté vers la critique.
- **Tristesse :** rythme lent, ton grave, introspection.
- **Joie :** rythme rapide, style fluide et enthousiaste.
- **Admiration :** langage figuré et ample.

Exemples

« Sans détour, dis-moi ce qui ne va pas dans cette stratégie. »
« Parle avec gravité des risques systémiques en trois points. »

À retenir : L'émotion structure le raisonnement en modulant l'énergie et la focalisation.

8. Le paradoxe pousse la nuance

Principe : Introduire une contradiction dans le prompt oblige le modèle à produire une réponse nuancée.

Effets observés

- Le modèle cherche un équilibre entre deux pôles logiques.
- Il produit une pensée dialectique plutôt qu'un simple commentaire.

Exemples

« Explique pourquoi l'IA est à la fois une menace et une promesse. »
« Montre en quoi le progrès peut être à la fois un risque et une chance. »

À retenir : Le paradoxe favorise la nuance et la réflexion véritable.

Limites actuelles des LLMs : deux axes majeurs

1. Hallucinations

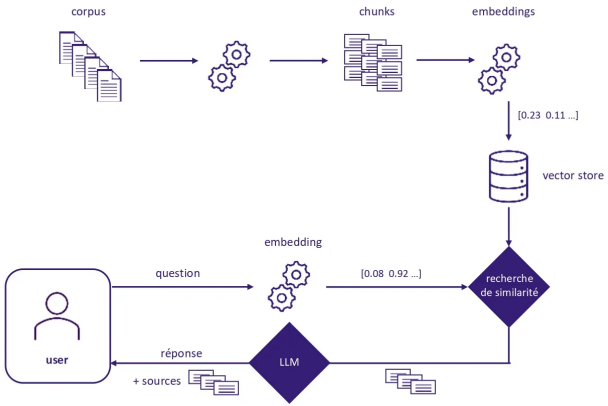
- Génération de contenus inexacts ou inventés, parfois avec une forte confiance.
- Difficulté à distinguer le vrai du plausible, surtout hors du domaine d'entraînement.
- Conséquences : risque de désinformation, perte de fiabilité, nécessité de vérification humaine.

2. Non-déterminisme

- Un même prompt peut produire plusieurs réponses différentes.
- Résultats sensibles à la température, au contexte ou à de légères variations de formulation.
- Enjeu pour la reproductibilité et le contrôle qualité des systèmes fondés sur les LLMs.

Retrieval Augmented Generation (RAG)

RAG basique


corpus
chunks
embeddings
[0.23 0.11 ...]
vector store
embedding
question
[0.08 0.92 ...]
recherche de similarité
user
réponse
+ sources
LLM

Stratégies de chunking

- **Définition** : Le chunking consiste à segmenter un document en morceaux (chunks) pour optimiser la récupération et la génération d'informations dans un système RAG.

- **Principales stratégies** :
  - **Fixed-size chunking** : découpage en segments de longueur fixe (ex : 512 tokens), simple mais peut fragmenter le contexte.
  - **Overlapping chunks** : chevauchement entre les segments pour conserver le contexte et éviter la perte d'informations critiques.
  - **Semantic chunking** : segmentation basée sur la structure du texte (paragraphes, titres, sections) ou l'analyse sémantique.
  - **Recursive chunking** : division progressive en fonction de la granularité des informations pertinentes.

- **Exemples d'outils** : LangChain, NLTK, SpaCy, tiktoken.

- **Avantages** : Améliore la récupération d'informations, réduit le bruit et optimise l'usage des modèles génératifs.

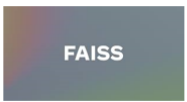
Bases de données vectorielles

- **Définition** : Stockent et indexent des représentations numériques (embeddings) pour effectuer des recherches sémantiques efficaces.

- **Principe** : Chaque document est transformé en un vecteur dense dans un espace de grande dimension. La recherche repose sur la similarité (cosinus, ANN, etc.) plutôt que sur des mots-clés.







Approches avancées du RAG

- **Reranking** : Amélioration de la sélection des documents récupérés en utilisant un modèle plus puissant (ex : cross-encoder, Cohere Rerank, ColBERT).
- **Optimisation du retrieveur** : Amélioration de l'indexation et du scoring des documents avec des données représentatives du domaine d'application.
- **Graph RAG** : Utilisation d'un graphe de connaissances pour structurer les relations entre documents et améliorer la récupération d'information.
- **Agent RAG** : Intégration d'agents conversationnels capables d'exécuter des actions (recherches multiples, vérification des réponses) avant de générer une réponse finale.

Reranking dans le RAG

- **Définition** : Le reranking consiste à réordonner les documents récupérés pour améliorer la pertinence des résultats avant de les passer au modèle génératif.

- **Méthodes principales** :
  - **BM25 + Reranker** : recherche lexicale suivie d'un modèle neuronal qui affine le classement des documents.
  - **Cross-encoder** : modèle BERT-like qui évalue chaque document en prenant en compte à la fois la requête et le contenu.
  - **Dense Reranking** : ajuste les scores des documents en utilisant des embeddings appris sur des données spécifiques.

- **Exemples d'outils** : Cohere Rerank, ColBERT, MonoT5, Rank-BERT.

- **Avantages** : Meilleure précision des résultats, réduction du bruit, amélioration des performances du modèle génératif.

Graph RAG

- **Définition** : Le Graph RAG utilise une structure de graphe pour organiser et récupérer l'information de manière plus contextuelle et structurée.

- **Principes clés** :
  - **Relations entre documents** : les nœuds représentent des documents, concepts ou entités, et les arêtes définissent leurs connexions.
  - **Recherche basée sur la connectivité** : la récupération d'informations se fait en explorant les relations entre les nœuds pour trouver les réponses les plus pertinentes.
  - **Augmentation du contexte** : le modèle peut utiliser des chemins sémantiques dans le graphe pour enrichir le prompt avec des données plus cohérentes.

- **Exemples d'outils** : Neo4j, NetworkX, Weaviate.

- **Avantages** : Meilleure structuration des connaissances, récupération d'information plus précise, réduction des erreurs contextuelles.

Agent RAG

- **Définition** : L'Agent RAG intègre un agent autonome qui orchestre la récupération et l'exploitation des documents pour affiner la génération de réponses.

- **Principes clés** :
  - **Multi-recherches adaptatives** : l'agent effectue plusieurs requêtes en fonction du contexte et ajuste dynamiquement le retrieveur.
  - **Vérification et correction** : l'agent peut analyser la réponse générée, détecter d'éventuelles erreurs et reformuler la requête si nécessaire.
  - **Actions spécifiques** : l'agent peut interagir avec des bases de données, des API externes ou exécuter du code pour enrichir la réponse finale.

- **Exemples d'outils** : LangChain Agents, CrewAI, etc.

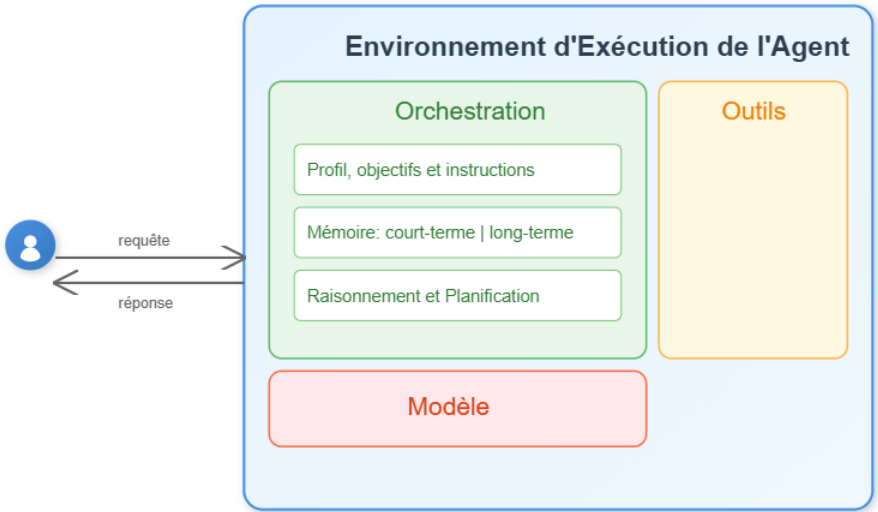
- **Avantages** : Plus grande autonomie, récupération d'informations plus dynamique, réduction des erreurs génératives.

Agents basés sur les LLM

Introduction aux Agents

- **Définition** : Les agents basés sur les LLMs sont des systèmes capables de prendre des actions de manière autonome en utilisant des outils mis à leur disposition.
- **Pourquoi les agents ?**
  Les modèles de langage seuls sont limités par leurs données d'entraînement. Les agents étendent ces capacités en utilisant des outils externes.
- **Objectif de la présentation :**
  - Expliquer le rôle des agents en IA.
  - Présenter leur architecture et leurs composants.
  - Explorer les cas d'utilisation et les techniques modernes d'orchestration.

Principe d'un Agent



Chain-of-Thought (CoT)

- **Qu'est-ce que CoT ?**
  Une technique de raisonnement permettant aux modèles de langage de diviser une tâche complexe en étapes intermédiaires.

- **Fonctionnement** :
  - Le modèle génère une séquence de raisonnements explicites avant de fournir une réponse.
  - Permet d'améliorer la précision et la transparence des décisions prises par un agent.

- **Applications** :
  - Résolution de problèmes mathématiques complexes.
  - Compréhension de texte avec inférences logiques.
  - Réponses plus précises et cohérentes dans les dialogues IA.

ReAct : Reasoning + Acting

Définition : ReAct est une approche qui combine le raisonnement et l'action pour améliorer la prise de décision des modèles de langage.

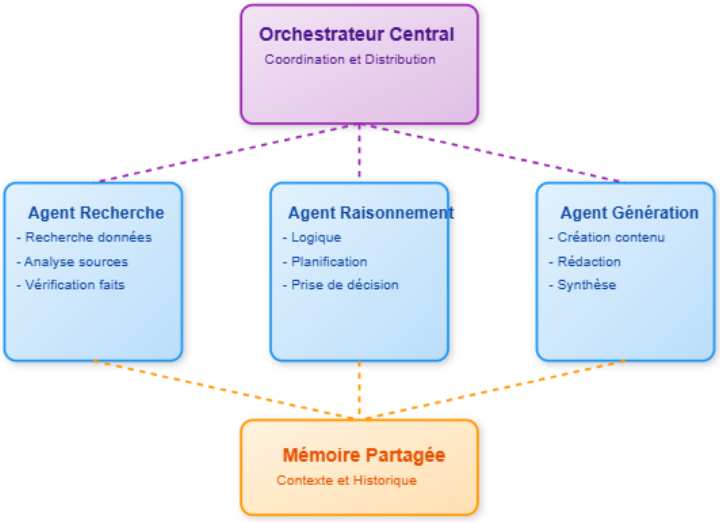
Principe :

- Le modèle observe une entrée utilisateur et génère une pensée explicite (**reasoning**).
- Il sélectionne une action (**acting**) basée sur cette pensée et l'exécute.
- Une observation est obtenue en réponse à l'action, servant de contexte pour la prochaine itération.
- Ce cycle de raisonnement et d'action continue jusqu'à atteindre une réponse satisfaisante.

Avantages :

- Réduction des hallucinations en intégrant des informations en temps réel.
- Amélioration de l'interprétabilité grâce aux traces de raisonnement.
- Meilleure capacité d'adaptation dans des environnements dynamiques.

Systèmes multiagents LLMs



Model Context Protocol (MCP)

Model Context Protocol (MCP) est un protocole client/serveur qui formalise l'accès d'un LLM à des capacités externes.

Rôles

- **LLM** : raisonnement, sélection d'actions
- **Client MCP** : orchestration, validation, politiques
- **Serveur MCP** : exposition et exécution des capacités

Principe fondamental :

> *le LLM ne fait que choisir dans un espace d'actions explicitement déclaré*

MCP ne modifie pas la capacité du LLM à raisonner ; il modifie la **structure** de ce qu'il est autorisé à faire.

Modèle formel : espace d'actions contraint

Soit un ensemble d'actions accessibles au LLM :

$$\mathcal{A} = \{a_1, \ldots, a_n\}$$

Chaque action a_i (tool) est définie par un contrat :

$$a_i = (\text{name}_i,\ \Sigma_i^{in},\ \Sigma_i^{out},\ \text{policy}_i)$$

- Σ^{in} : schéma d'entrée (types, contraintes, champs requis)
- Σ^{out} : schéma de sortie (résultat, erreurs)
- `policy` : permissions, scopes, budgets

Le LLM doit produire un couple valide (a_i, x) avec $x \in \Sigma_i^{in}$. Toute sortie hors contrat est non exécutable.

Briques MCP : Resources, Tools, Prompts

Un serveur MCP expose trois catégories distinctes :

Resources

- Contexte adressable (URI)
- Lecture contrôlée : fichiers, pages web, tables, logs

Tools

- Fonctions exécutables
- Interfaces strictement typées (JSON Schema)
- Effets réels : I/O, APIs, automatisation

Prompts

- Patrons de raisonnement paramétrés
- Standardisation du comportement entre agents

MCP expose des contrats plutôt que des implémentations.

Cycle d'exécution MCP (raisonnement → action)

Séquence minimale :

1. **Discovery**
   - `list_tools`, `list_resources`, `list_prompts`
2. **Sélection**
   - Le LLM choisit un tool et fournit des arguments conformes au schéma
3. **Validation**
   - Le client valide types, contraintes, policies
4. **Exécution**
   - Le serveur exécute et retourne un résultat structuré

La robustesse du système repose sur la validité des schémas, pas sur l'interprétation du texte.

Ce que MCP apporte réellement

Apports

- Explicitation de l'espace d'actions
- Séparation décision / exécution
- Gouvernance par construction (permissions, budgets)
- Auditabilité via les appels de tools

Non-apports

- Pas plus d'intelligence
- Pas plus d'autonomie
- Pas d'exécution dans le modèle

MCP ne rend pas les LLM plus puissants ; il rend les systèmes agentiques plus maîtrisables.

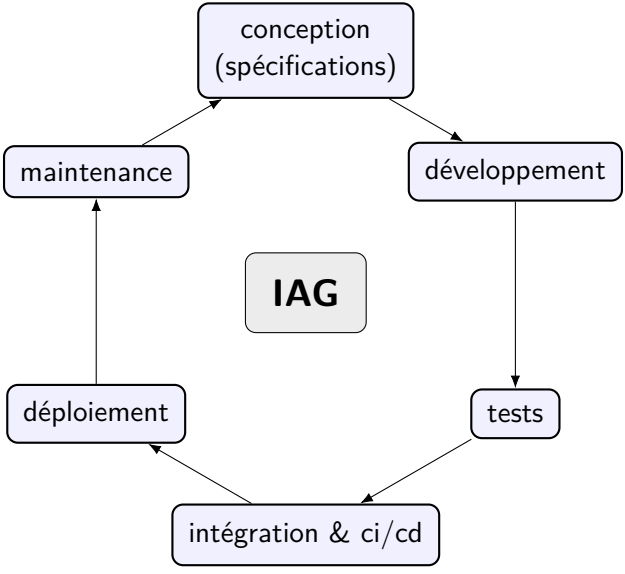
IAG dans le cycle de développement

L'IAG dans le cycle de vie logiciel

L'intelligence artificielle générative ne se limite pas à l'autocomplétion de code : elle peut s'intégrer à chaque étape du cycle de vie logiciel pour assister les équipes.

- **Conception** : clarification des besoins, spécifications, ébauches d'architecture.
- **Développement** : génération et refactor de code, assistance aux patterns.
- **Tests** : génération de tests unitaires et fonctionnels, détection de cas limites.
- **Intégration & CI/CD** : review automatique de PR, génération de changelog et documentation.
- **Déploiement** : notes de version, assistance à l'infrastructure-as-code.
- **Maintenance** : classification des tickets, suggestions de correctifs, documentation à partir du code.

IAG dans le cycle de vie logiciel



Conception et spécifications

Objectif : réduire l'ambiguïté et accélérer le passage du besoin métier à la solution technique.

- Transformer une *user story* en endpoints d'API (routes, verbes, schémas JSON).
- Générer une première ébauche d'architecture (services, flux, dépendances).
- Produire un glossaire ou des critères d'acceptation testables.

Exemple de prompt : *« À partir de cette user story, propose les endpoints REST, les codes d'erreur et un schéma JSON pour POST /orders. »*

Développement

Objectif : accélérer l'implémentation tout en améliorant la lisibilité et la maintenabilité.

- Génération de fonctions à partir d'une description en langage naturel.
- Refactor : extraction de sous-fonctions, renommage de variables, simplification de structures.
- Suggestions de design patterns adaptés au contexte.

Exemple : Avant : fonction monolithique de 80 lignes. Après : découpage en `validate()`, `transform()`, `persist()` avec docstrings et exceptions typées.

Tests

Objectif : améliorer la couverture et détecter les cas limites dès la phase de développement.

- Génération de tests unitaires directement à partir du code.
- Proposition de cas extrêmes : valeurs limites, erreurs réseau, formats inattendus.
- Création de scénarios fonctionnels dérivés des user stories.

Exemple de prompt : *« Voici la fonction price(order). Génère 8 tests unitaires `pytest` couvrant remise, TVA, devise inconnue, quantité négative, seuils, valeur nulle. »*

Intégration et CI/CD

Objectif : enrichir les pipelines d'intégration continue avec des capacités d'analyse et de génération.

- Review automatique de pull requests : détection de failles de sécurité, de duplications, de style incohérent.
- Génération automatique de tests complémentaires pour valider les commits.
- Production de documentation et de changelogs à partir des messages de commits.

Exemple : Un workflow GitHub Actions envoie le *diff* à un LLM, qui commente directement la PR avec ses suggestions.

Déploiement

Objectif : automatiser et simplifier les tâches de mise en production.

- Assistance à la rédaction de scripts *infrastructure-as-code* (Terraform, Ansible).
- Génération automatique des notes de version lisibles à partir des commits.
- Support aux équipes DevOps pour documenter et communiquer les changements.

Exemple : À partir de commits techniques, l'IAG rédige un changelog clair pour les utilisateurs finaux.

Maintenance et support

Objectif : accélérer la résolution de problèmes et réduire la dette technique.

- Classification automatique des tickets d'incidents par priorité et catégorie.
- Proposition de correctifs initiaux pour certains bugs.
- Génération de documentation à partir de code existant ou de logs.

Exemple : Un log d'erreur est soumis à l'IAG, qui propose un diagnostic et une piste de correction.

Les fichiers AGENTS.md et SKILL.md

AGENTS.md

Consignes propres au projet : conventions, commandes de test et règles de contribution.

`projet/AGENTS.md`

```
# Validation
- Exécuter les tests : pytest
- Ajouter un test de régression
pour chaque bug corrigé.
```

SKILL.md

Procédure spécialisée, chargée selon la tâche. Elle peut inclure des scripts et des références.

`revue-code/SKILL.md`

```
---
name: revue-code
description: Relire le code.
---
Examiner le diff et les tests.
```

L'agent applique les consignes et exécute les actions avec ses outils.

Quatre dépôts de skills à connaître

Des procédures prêtes à l'emploi, à choisir selon la tâche.

Addy Osmani addyosmani/agent-skills	Cadrer, développer et vérifier un projet. Spécifications, tests et revue de code.
Anthropic anthropics/skills	Créer et traiter des documents. PDF, Word, Excel et PowerPoint.
Vercel Labs vercel-labs/agent-skills	Améliorer les applications React / Next.js. Performance et qualité des interfaces.
Superpowers obra/superpowers	Organiser le travail d'un agent développeur. Planification, tests et débogage méthodique.

Enjeux éthiques, sécurité et conformité de l'IAG

Pourquoi parler d'éthique et de sécurité ?

L'intelligence artificielle générative ne pose pas seulement des défis techniques : elle soulève des enjeux critiques pour l'entreprise.

- **Sécurité** : risques de fuite de secrets ou d'attaques via les prompts.
- **Confidentialité** : conformité au RGPD, respect des données sensibles.
- **Propriété intellectuelle : propriété des données d'entraînement et du contenu généré par l'IA.**
- **Éthique** : biais, équité, responsabilité dans l'usage.

Message clé : l'adoption de l'IAG doit être accompagnée d'une gouvernance adaptée.

Sécurité de l'IA générative

- **Fuite de données** : transmission de documents confidentiels, de code propriétaire ou d'identifiants à un service externe ; divulgation d'informations à un utilisateur non autorisé.
- **Injection de prompts** : des instructions malveillantes, présentes dans une requête, un document ou une page web, cherchent à détourner le comportement du système.
- **Résultats dangereux** : génération de code vulnérable ou de commandes risquées, ensuite utilisés sans vérification suffisante.
- **Actions non maîtrisées** : un système connecté à des outils peut modifier, supprimer ou transmettre des données au-delà de ce qui était attendu.

Mesures de protection : limiter les accès et les permissions, protéger les secrets, vérifier les résultats avant exécution, faire valider les actions sensibles et prévoir une journalisation ainsi qu'un mécanisme d'arrêt.

Enjeux de confidentialité

Problématique centrale : l'IAG traite souvent des données sensibles ou personnelles.

- **RGPD** : droit à l'oubli, consentement, minimisation des données.
- **Localisation des données** : datacenters UE vs hors UE.
- **Solutions** :
  - anonymisation avant envoi,
  - hébergement de modèles open source en interne,
  - passage par un proxy d'entreprise.

Propriété intellectuelle et IA générative

- **Données d'entraînement** : l'utilisation d'œuvres protégées soulève des questions de respect des droits, de transparence sur les sources et de rémunération des créateurs.
- **Protection des contenus générés** : en France et dans l'Union européenne, la protection par le droit d'auteur repose sur des choix créatifs humains originaux. Utiliser une IA ne confère donc pas automatiquement un droit d'auteur sur le résultat.
- **Risque de reproduction** : un texte, une image ou du code généré peut reprendre des éléments protégés. La génération par IA ne garantit pas que le contenu soit librement réutilisable.
- **Licences et contrats** : distinguer les conditions d'utilisation du modèle, les droits sur les contenus fournis et les conditions d'exploitation des résultats.

Bonnes pratiques : vérifier les droits et licences applicables, conserver les sources et les contributions humaines, examiner les contenus avant leur publication ou leur livraison.

Biais et IA générative

- **Reproduction des stéréotypes** : les modèles peuvent reproduire ou amplifier les préjugés sociaux, culturels ou de genre présents dans leurs données d'entraînement.
- **Traitements discriminatoires** : des situations comparables peuvent recevoir des réponses différentes selon des caractéristiques personnelles ou des indices indirects, comme un prénom, une adresse ou une manière de s'exprimer.
- **Performances inégales** : certaines langues, populations ou situations sont moins bien représentées dans les données. Une bonne performance moyenne peut masquer des erreurs plus fréquentes pour certains utilisateurs.
- **Alignement et injection explicite de valeurs** : l'alignement et les instructions du système orientent les réponses selon des normes et des valeurs choisies par les concepteurs ou les organisations qui déploient le modèle. Ces choix ne sont pas nécessairement partagés par tous les utilisateurs.

Responsabilité et IA générative

- **Dilution de la responsabilité** : en cas d'erreur, chaque intervenant peut renvoyer la responsabilité vers un autre (fournisseur du modèle, organisation ou utilisateur) sans que personne n'assume les conséquences.
- **Confiance excessive** : une réponse convaincante peut être acceptée sans vérification. La validation humaine devient alors une formalité qui laisse passer les erreurs.
- **Décisions difficiles à justifier** : l'absence de traces sur les informations utilisées et les arbitrages effectués complique l'explication d'une décision et l'identification des causes d'un dommage.

Bonnes pratiques : clarifier les responsabilités, organiser une supervision humaine réelle, documenter les décisions importantes et prévoir des procédures de contestation et de correction.

Développement durable et IA générative

- **Consommation énergétique** : l'entraînement des modèles et leur utilisation mobilisent des ressources de calcul. Les émissions associées dépendent notamment du volume d'usage, de l'efficacité des infrastructures et de l'origine de l'électricité.
- **Empreinte matérielle** : la fabrication des puces, des serveurs et des infrastructures mobilise des matières premières. Le renouvellement des équipements accroît les besoins d'extraction et la production de déchets électroniques.
- **Effet rebond** : des modèles plus efficaces et moins coûteux peuvent encourager une multiplication des usages. Les gains obtenus par requête peuvent alors être compensés, voire dépassés, par l'augmentation du volume total.

Bonnes pratiques : privilégier des modèles légers lorsque la tâche le permet, limiter les tokens d'entrée et de sortie au nécessaire, éviter les appels redondants et réutiliser les résultats pertinents, tout en maintenant la qualité attendue.

Emploi, travail et compétences

- **Fragilisation de l'emploi** : l'automatisation de certaines tâches peut réduire les besoins sur certains postes et transformer les compétences recherchées, avec des conséquences différentes selon les métiers et les qualifications.
- **Perte de compétences** : une délégation excessive peut affaiblir la capacité à comprendre, produire et vérifier un travail. Pour les débutants, elle peut aussi supprimer des occasions d'apprentissage indispensables à la construction de l'expertise.
- **Intensification du travail** : les gains de temps peuvent se traduire par une augmentation des objectifs et des cadences, plutôt que par une amélioration des conditions de travail.
- **Partage inégal des gains** : les bénéfices de productivité peuvent être concentrés chez certains acteurs, tandis que les salariés supportent les efforts d'adaptation et les risques de remplacement.

Souveraineté et IA générative

- **Dépendance technologique** : le recours à des fournisseurs externes peut rendre une organisation dépendante de leurs décisions sur les prix, les fonctionnalités, les conditions d'utilisation et la continuité des services.
- **Verrouillage des solutions** : des intégrations propriétaires et des investissements spécifiques peuvent rendre le changement de fournisseur coûteux et complexe, réduisant la liberté de choix.
- **Maîtrise des actifs stratégiques** : externaliser le traitement des données, du code et des connaissances métier peut réduire le contrôle de l'organisation sur son patrimoine informationnel et accroître sa dépendance à des compétences externes.
- **Dépendance géopolitique et industrielle** : l'accès aux modèles, aux infrastructures de calcul et aux composants peut être affecté par des décisions politiques, des restrictions commerciales ou des tensions internationales.

Merci de votre attention

redha.moulla@axia-conseil.com